Gradient Descent for Beginners

Ideas and Intuition

Shengqin Jiang | 2026.03

Agenda

1. Introduction
2. Limitations and Challenges of Regression
3. Loss Surface and Finding Optimal Solutions
4. Mathematical Logic of Gradient Descent
5. Intuitive Understanding of Backpropagation
6. From Univariate to Multivariate Derivation
7. High-Dimensional Features and Optimization Strategies
8. Nonlinear Transmission Chains and Activation Functions

01 Introduction

CO_2 emissions are one of the main causes of global warming, and vehicles are major emission sources.

- Goal: Build a model that inputs vehicle features (engine size, fuel consumption, etc.) and outputs predicted CO_2 emissions
- Dataset: FuelConsumptionCo2.csv (public dataset from the Government of Canada)

Interactive: Requirement Analysis

Open the interactive page, explore the data, reflect on the meaning and challenges of prediction, and intuitively understand the importance of data prediction in the real world.

02 Limitations and Challenges of Regression

High School Statistics vs. Test Set Validation

Fitting data with simple regression and observing performance on the test set reveals:

- Traditional statistical methods encounter bottlenecks when handling complex data
- Computational speed is severely limited

Interactive: Model Assumptions

Open the interactive page and observe the fitting performance of the linear model using least squares regression:

- The model is affected by multiple features, leading to unsatisfactory fitting
- We need to add features to increase model complexity and improve prediction accuracy

03 Loss Surface and Finding Optimal Solutions

What is “Loss”?

The loss function measures the difference between predicted values and true values. Smaller loss means better prediction.

- Mean Squared Error (MSE): $L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
- Mean Absolute Error (MAE): $L = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$
- Binary Cross-Entropy: $L = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right]$
- Categorical Cross-Entropy: $L = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{i,k}\log(\hat{y}_{i,k})$

Interactive 3: Visual Understanding

Open the interactive page, manually adjust parameters k and b to find the minimum of the loss function, and observe model fitting.

Thinking & Exploration

Can we design an algorithm to find the minimum of the loss function for fitting, instead of brute-force solving?

04 Mathematical Logic of Gradient Descent

Definition of Gradient

- **Gradient** is the vector of all first partial derivatives of a function

The derivative of a function at a point represents the direction and rate of function change.

Interactive: Parameter Update Demo

Open the interactive page, adjust the learning rate, observe the trajectory of the ball on the loss surface, and intuitively understand how learning rate affects convergence speed and stability.

05 Intuitive Understanding of Backpropagation

Core Definition of Backpropagation

Backpropagation is an efficient method for computing gradients.

It calculates the partial derivatives of the loss function with respect to each parameter layer by layer, from output to input.

It uses the **chain rule** to decompose complex derivatives and efficiently compute gradients for network parameters.

Descent Process

Update the parameter x iteratively along the **negative gradient direction** until reaching the minimum point.

$$Loss = \frac{x^2}{2}$$

$$Gradient = \frac{d}{dx} \cdot \frac{x^2}{2} = x$$

```
lr = 0.1
x = 2
for _ in range(100):
    Loss = (x ** 2) / 2 # Loss function
    Gradient = x # Gradient
    x = x - lr * Gradient # Update param
print(f'x:{x:.2f}; Loss:{Loss:.2f}')
```

06 From Univariate to Multivariate Derivation

1. Extension of scalar computation rules
2. Gradient descent for multivariate features

Extension of scalar computation rules

$$[w_1, w_2, \ldots, w_n] \cdot \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + b = C$$

$$\begin{bmatrix} w_{11} & w_{12} & \ldots & w_{1n} \\ w_{21} & w_{22} & \ldots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \ldots & w_{mn} \end{bmatrix} \cdot \begin{bmatrix} x_{11} & x_{12} & \ldots & x_{1k} \\ x_{21} & x_{22} & \ldots & x_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \ldots & x_{nk} \end{bmatrix} + \begin{bmatrix} b_{11} & b_{12} & \ldots & b_{1k} \\ b_{21} & b_{22} & \ldots & b_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & \ldots & b_{mk} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & \ldots & c_{1k} \\ c_{21} & c_{22} & \ldots & c_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ c_{m1} & c_{m2} & \ldots & c_{mk} \end{bmatrix}$$

Extension of scalar computation rules

1. Scalar expansion

$$WX + b = C$$
$$w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = C$$

2. Derivative with respect to a single weight

$$\frac{dC}{dw_n} = \frac{d}{dw_n}(w_1x_1 + w_2x_2 + \cdots + w_nx_n + b) = x_n$$

3. Combine into matrix form

$$\frac{dC}{dW} = [x_1, x_2, \ldots, x_n]^T = X^T$$

4. Derivative with respect to bias

$$\frac{dC}{db} = \frac{d}{db}(w_1x_1 + w_2x_2 + \cdots + w_nx_n + b) = 1$$

Extension of scalar computation rules

$$WX + B = C$$

1. Scalar expansion

$$c_{ij} = \sum_{t=1}^{n} w_{it} x_{tj} + b_{ij} \quad (i = 1, 2, \ldots, m; j = 1, 2, \ldots, k)$$

2. Derivative with respect to a single value

$$\frac{dC}{dw_{pq}} = \frac{d}{dw_{pq}} \left(\sum_{t=1}^{n} w_{it} x_{tj} + b_{ij} \right) = x_{qj}; \frac{dC}{db_{pq}} = \frac{d}{db_{pq}} \left(\sum_{t=1}^{n} w_{it} x_{tj} + b_{ij} \right) = 1$$

3. Combine into matrix form

$$\frac{dC}{dW} = [x_1, x_2, \ldots, x_n]^T = X^T; \frac{dB}{dW} = [x_1, x_2, \ldots, x_n]^T = \mathbf{1}_{m \times k}$$

Gradient descent for multivariate features

$$Y_{pred} = W \cdot X + B$$

$$L = \text{MSE} = \frac{1}{2}(Y_{true} - Y_{pred})^2$$

$$\frac{dL}{dY_{pred}} = \frac{d}{dY_{pred}}\left[\frac{1}{2}(Y_{true} - Y_{pred})^2\right] = -(Y_{true} - Y_{pred})$$

$$\frac{dY_{pred}}{dW} = \frac{d}{dW}(W \cdot X + B) = X^T$$

$$\frac{dY_{pred}}{dB} = \frac{d}{dB}(W \cdot X + B) = 1$$

Final Gradient (Chain Rule)

$$\frac{dL}{dW} = \frac{dL}{dY_{pred}} \cdot \frac{dY_{pred}}{dW} = -(Y_{true} - Y_{pred}) \cdot X^T = (Y_{pred} - Y_{true}) \cdot X^T$$

$$\frac{dL}{dB} = \frac{dL}{dY_{pred}} \cdot \frac{dY_{pred}}{dB} = -(Y_{true} - Y_{pred}) \cdot 1 = Y_{pred} - Y_{true}$$

Gradient descent for multivariate features and samples

$$Y_{pred}^{(i)} = W \cdot X^{(i)} + B$$

$$L = \text{MSE} = \frac{1}{2n} \sum_{i=1}^{n} (Y_{true}^{(i)} - Y_{pred}^{(i)})^2$$

$$\frac{dL}{dY_{pred}^{(i)}} = \frac{d}{dY_{pred}^{(i)}} \left[\frac{1}{2n} \sum_{i=1}^{n} (Y_{true}^{(i)} - Y_{pred}^{(i)})^2 \right] = -\frac{1}{n} (Y_{true}^{(i)} - Y_{pred}^{(i)})$$

$$\frac{dY_{pred}^{(i)}}{dW} = \frac{d}{dW} (W \cdot X^{(i)} + B) = (X^{(i)})^T$$

$$\frac{dY_{pred}^{(i)}}{dB} = \frac{d}{dB} (W \cdot X^{(i)} + B) = 1$$

Final Gradient (Chain Rule)

$$\frac{dL}{dW} = \sum_{i=1}^{n} \left[-\frac{1}{n} (Y_{true}^{(i)} - Y_{pred}^{(i)}) \cdot (X^{(i)})^T \right] = \frac{1}{n} \sum_{i=1}^{n} \left[(Y_{pred}^{(i)} - Y_{true}^{(i)}) \cdot (X^{(i)})^T \right]$$

$$\frac{dL}{dB} = \sum_{i=1}^{n} \left[-\frac{1}{n} (Y_{true}^{(i)} - Y_{pred}^{(i)}) \cdot 1 \right] = \frac{1}{n} \sum_{i=1}^{n} \left(Y_{pred}^{(i)} - Y_{true}^{(i)} \right)$$

07 High-Dimensional Features and Optimization Strategies

In reality, vehicle emissions are affected by many features: fuel consumption, engine displacement, number of cylinders, fuel type, weight, etc.

With more than 2 features, we enter high-dimensional space.

Challenges in High-Dimensional Space

1. Explosion in the number of parameters → higher computation and memory cost, slower convergence
2. Complex loss surface → saddle points, flat regions, local optima, and easy trapping

SGD – Stochastic Gradient Descent

- Core: Compute gradients using single or mini-batch samples instead of the full dataset
- Advantages: Fast training, low memory, suitable for big data / high dimensions; noise helps escape local optima
- Disadvantages: Noisy gradient estimates, unstable updates, oscillating convergence

Adam – Adaptive Moment Estimation

- Combines momentum (first-order moment) and adaptive learning rates (second-order moment)
- Dynamically adjusts learning rates per parameter using exponential moving averages of gradients and squared gradients
- Advantages: Fast and stable convergence; adaptive learning rates reduce manual tuning; robust to sparse gradients/noisy data; works well for non-convex problems and large-scale deep learning

1. $\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\nabla_\theta Loss_t$
2. $\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)(\nabla_\theta Loss_t)^2$
3. $\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}$
4. $\theta_t = \theta_{t-1} - \eta \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t}+\epsilon}$

PCA – Principal Component Analysis

A classic unsupervised dimensionality reduction method that compresses high-dimensional data into low dimensions while preserving as much information as possible.

- Finds directions of maximum variance as principal components, removes redundant and highly correlated dimensions
- Alleviates the curse of dimensionality, reduces computation, eliminates feature correlation, smoothes the loss surface, and enables high-dimensional data visualization

08 Nonlinear Transmission Chains and Activation Functions

Limitations of Linear Models

Using only linear transformations ($y = wx + b$), no matter how many layers are stacked, the entire model remains a linear combination of inputs.
It cannot model most real-world nonlinear problems: image edge detection, speech tone variation, financial data fluctuations, etc.

Role of Activation Functions

An activation function is a **nonlinear mapping** applied to neuron outputs, breaking the limitation of stacked linear transformations.
After applying activation functions such as Sigmoid and ReLU, the output is no longer linear in the input.
Multi-layer networks then gain the ability to fit any complex nonlinear functions.

Sigmoid function

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$\sigma'(z) = \frac{d}{dz}\left(\frac{1}{1+e^{-z}}\right) = \frac{e^{-z}}{(1+e^{-z})^2}$$

$$\sigma'(z) = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z) \cdot \big(1-\sigma(z)\big)$$

$$\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)$$

ReLU function

$$\mathrm{ReLU}(z) = \max(0, z)$$

$$\mathrm{ReLU}'(z) = \frac{d}{dz}(\max(0, z)) = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \\ \text{undefined}, & z = 0 \end{cases}$$

$$\mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

Tanh function

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\tanh'(z) = \frac{d}{dz}\left(\frac{e^z - e^{-z}}{e^z + e^{-z}}\right) = \frac{(e^z + e^{-z})^2 - (e^z - e^{-z})^2}{(e^z + e^{-z})^2}$$

$$\tanh'(z) = \frac{4}{(e^z + e^{-z})^2} = 1 - \left(\frac{e^z - e^{-z}}{e^z + e^{-z}}\right)^2$$

$$\tanh'(z) = 1 - \tanh^2(z)$$

Softmax function

$$\sigma_i(\boldsymbol{z}) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \quad (i = 1, 2, \ldots, K)$$

$$\frac{d\sigma_i}{dz_k} = \frac{d}{dz_k}\left(\frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}\right) = \begin{cases} \frac{e^{z_i}\left(\sum_{j=1}^{K} e^{z_j} - e^{z_i}\right)}{\left(\sum_{j=1}^{K} e^{z_j}\right)^2}, & i = k \\ -\frac{e^{z_i} e^{z_k}}{\left(\sum_{j=1}^{K} e^{z_j}\right)^2}, & i \neq k \end{cases}$$

$$\frac{d\sigma_i}{dz_k} = \begin{cases} \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \cdot \left(1 - \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}\right), & i = k \\ -\frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \cdot \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, & i \neq k \end{cases}$$

$$\frac{d\sigma_i}{dz_k} = \begin{cases} \sigma_i(\boldsymbol{z}) \cdot \big(1 - \sigma_i(\boldsymbol{z})\big), & i = k \\ -\sigma_i(\boldsymbol{z}) \cdot \sigma_k(\boldsymbol{z}), & i \neq k \end{cases}$$

DG Task 1

$$Z_1 = XW_1 + b_1, \quad A_1 = \sigma(Z_1)$$
$$Z_2 = A_1W_2 + b_2, \quad A_2 = \sigma(Z_2)$$
$$Z_3 = A_2W_3 + b_3, \quad \hat{y} = \sigma(Z_3)$$
$$Loss = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$$

DG Task 1 Solution

1. First Derive the Error Terms between the layers

- Output layer error term: $\delta_3 = (\hat{y} - y) \cdot \sigma'(Z_3) = (A_3 - y) \cdot A_3(1 - A_3)$

```
a2_ = Loss_*a[2]*(1-a[2])
```

- Hidden layer 2 error term: $\delta_2 = \delta_3 \cdot W_2^T \cdot \sigma'(Z_2) = \delta_3 \cdot W_2^T \cdot A_2(1 - A_2)$

```
a1_ = np.dot(a2_, w[2].T)*a[1]*(1-a[1])
```

- Hidden layer 1 error term: $\delta_1 = \delta_2 \cdot W_1^T \cdot \sigma'(Z_1) = \delta_2 \cdot W_1^T \cdot A_1(1 - A_1)$

```
a0_ = np.dot(a1_, w[1].T)*a[0]*(1-a[0])
```

DG Task 1 Solution

2. Gradient Formulas for Each Parameter (Core)

(1) Gradient of W_2 (w[2]) :$\nabla_{W_2} Loss = \frac{1}{m} A_2^T \cdot \delta_3$

Weight update in code: $W_2 = W_2 - \eta \cdot \frac{1}{m} A_2^T \cdot \delta_3$

```
w[2] = w[2] - lr*np.dot(a[1].T, a2_)/l
```

(2) Gradient of b_2 (b[2]) :$\nabla_{b_2} Loss = \frac{1}{m} \sum_{i=1}^{m} \delta_{3(i)}$ (summation is row-wise)

Bias update in code: $b_2 = b_2 - \eta \cdot \frac{1}{m} \sum \delta_3$

```
b[2] = b[2] - lr*np.sum(a2_, axis=0, keepdims=True)/l
```

DG Task 1 Solution

2. Gradient Formulas for Each Parameter (Core)

(3) Gradient of W_1 (w[1]) :$\nabla_{W_1} Loss = \frac{1}{m} A_1^T \cdot \delta_2$

Weight update in code: $W_1 = W_1 - \eta \cdot \frac{1}{m} A_1^T \cdot \delta_2$

```
w[1] = w[1] - lr*np.dot(a[0].T, a1_)/l
```

(4) Gradient of b_1 (b[1])$:\nabla_{b_1} Loss = \frac{1}{m} \sum_{i=1}^m \delta_{2(i)}$

Bias update in code: $b_1 = b_1 - \eta \cdot \frac{1}{m} \sum \delta_2$

```
b[1] = b[1] - lr*np.sum(a1_, axis=0, keepdims=True)/l
```

DG Task 1 Solution

2. Gradient Formulas for Each Parameter (Core)

(5) Gradient of W_0 (w[0]) :$\nabla_{W_0} Loss = \frac{1}{m} X^T \cdot \delta_1$

Weight update in code: $W_0 = W_0 - \eta \cdot \frac{1}{m} X^T \cdot \delta_1$

```
w[0] = w[0] - lr*np.dot(data.T, a0_)/l
```

(6) Gradient of b_0 (b[0]) :$\nabla_{b_0} Loss = \frac{1}{m} \sum_{i=1}^{m} \delta_{1(i)}$

Bias update in code: $b_0 = b_0 - \eta \cdot \frac{1}{m} \sum \delta_1$

```
b[0] = b[0] - lr*np.sum(a0_, axis=0, keepdims=True)/l
```

DG Task 2

$$Z_0 = XW_0 + b_0, \quad A_0 = \tanh(Z_0)$$
$$Z_1 = A_0W_1 + b_1, \quad A_1 = \mathrm{ReLU}(Z_1)$$
$$Z_2 = A_1W_2 + b_2, \quad \hat{y} = \mathrm{Softmax}(Z_2)$$
$$Loss = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{3} y_{i,k}\log(\hat{y}_{i,k} + \epsilon)$$

DG Task 2 Solution

1. First Derive the Error Terms between the layers

- Output layer error term (Softmax + Cross-Entropy): $\delta_2 = \hat{y} - y = A_2 - y$

```
d_2 = a[2] - res
```

- Hidden layer 2 error term (ReLU): $\delta_1 = \delta_2 \cdot W_2^T \cdot \sigma'(Z_1) = \delta_2 \cdot W_2^T \cdot \mathbb{I}(Z_1 > 0)$

```
relu_d = np.where(z[1] > 0, 1, 0)
d_1 = np.dot(d_2, w[2].T) * relu_d
```

- Hidden layer 1 error term (Tanh): $\delta_0 = \delta_1 \cdot W_1^T \cdot \sigma'(Z_0) = \delta_1 \cdot W_1^T \cdot (1 - A_0^2)$

```
tanh_d = 1 - np.square(a[0])
d_0 = np.dot(d_1, w[1].T) * tanh_d
```

DG Task 2 Solution

2. Gradient Formulas for Each Parameter (Core)

(1) Gradient of W_2 (w[2]) :$\nabla_{W_2} Loss = \frac{1}{m} A_1^T \cdot \delta_2$

Weight update in code: $W_2 = W_2 - \eta \cdot \frac{1}{m} A_1^T \cdot \delta_2$

```
w[2] -= lr * np.dot(a[1].T, d_2) / l
```

(2) Gradient of b_2 (b[2]) :$\nabla_{b_2} Loss = \frac{1}{m} \sum_{i=1}^{m} \delta_{2(i)}$ (summation is row-wise)

Bias update in code: $b_2 = b_2 - \eta \cdot \frac{1}{m} \sum \delta_2$

```
b[2] -= lr * np.sum(d_2, axis=0, keepdims=True)/l
```

DG Task 2 Solution

2. Gradient Formulas for Each Parameter (Core)

(3) Gradient of W_1 (w[1]) :$\nabla_{W_1} Loss = \frac{1}{m} A_0^T \cdot \delta_1$

Weight update in code: $W_1 = W_1 - \eta \cdot \frac{1}{m} A_0^T \cdot \delta_1$

```
w[1] -= lr * np.dot(a[0].T, d_1) / l
```

(4) Gradient of b_1 (b[1]) :$\nabla_{b_1} Loss = \frac{1}{m} \sum_{i=1}^{m} \delta_{1(i)}$

Bias update in code: $b_1 = b_1 - \eta \cdot \frac{1}{m} \sum \delta_1$

```
b[1] -= lr * np.sum(d_1, axis=0, keepdims=True)/l
```

DG Task 2 Solution

2. Gradient Formulas for Each Parameter (Core)

(5) Gradient of W_0 (w[0]) :$\nabla_{W_0} Loss = \frac{1}{m} X^T \cdot \delta_0$

Weight update in code: $W_0 = W_0 - \eta \cdot \frac{1}{m} X^T \cdot \delta_0$

```
w[0] -= lr * np.dot(data.T, d_0) / l
```

(6) Gradient of b_0 (b[0]) :$\nabla_{b_0} Loss = \frac{1}{m} \sum_{i=1}^{m} \delta_{0(i)}$

Bias update in code: $b_0 = b_0 - \eta \cdot \frac{1}{m} \sum \delta_0$

```
b[0] -= lr * np.sum(d_0, axis=0, keepdims=True)/l
```

08 Interactive Exploration of Neural Network Hyperparameters

Free Exploration on TensorFlow Playground

- TensorFlow Playground offers a visual, interactive environment to experiment with core neural network parameters, bridging the gap between theory and practice.
- By freely modifying activation function, tuning learning rate values, and observing real-time gradient descent trajectories, you can directly witness how each choice alters convergence speed, loss reduction, and model stability.
- This hands-on exploration builds an intuitive understanding of hyperparameter interactions that static learning cannot replicate.

Thank you

Shengqin Jiang | 2026.03